

UE03 Données et Analytics

Observabilité et détection de comportements sur un jeu multijoueur web

Modalités : travail individuel, 4 jours.

IA générative interdite (code, configuration, requêtes, rédaction). Tout usage avéré entraîne la note de 0. Chaque étudiant peut être interrogé à l'oral sur n'importe quelle partie de son rendu.

1. Contexte

Un studio indépendant exploite en bêta un jeu de tir multijoueur jouable dans le navigateur (parties de 1 ou 2 joueurs contre des bots, plusieurs cartes). Le client de jeu envoie des rapports lorsqu'une image met trop de temps à s'afficher ; le serveur journalise la fin de chaque partie.

Il n'existe aujourd'hui ni pipeline, ni centralisation des logs, ni métriques. La direction pose deux questions :

« Des joueurs se plaignent de saccades depuis jeudi soir. Pourquoi, et qui est touché ? » « Est-ce que quelqu'un abuse du jeu ? »

Vous êtes chargé de mettre en place la chaîne DevOps et d'observabilité, puis de répondre à ces questions avec des preuves.

2. Fournitures

- `game-telemetry-demo` : service Node.js (API d'ingestion, simulation de la flotte de parties, logs JSON lines, tests, lint, générateur de charge). Voir son README.
- `data/admin-export-2026-09-20_26.log` : export brut de 7 jours de production depuis la console d'administration.

3. Travail demandé

Partie A : intégration et déploiement continu

1. Dockerfile multi-étapes, image minimale, exécution sans root.
2. Pipeline (GitHub Actions ou GitLab CI) : lint, tests, construction de l'image, analyse de vulnérabilités, publication dans un registre avec un tag traçable.
3. Le pipeline échoue si les tests échouent ou si une vulnérabilité critique est détectée. Démontrez les deux cas.
4. Déploiement par `docker compose` : une seule commande lance le service, le générateur de charge et toute la stack d'observabilité.

Partie B : collecte de logs

1. Centralisez les logs en direct du service (Loki avec Promtail ou Alloy ; une autre stack est acceptée si elle est justifiée). Choix des labels justifié au regard de la cardinalité.
2. Ingérez l'export historique. Il est brut : à vous d'en identifier les pièges (format, horodatage, fuseau, qualité des données) et de les traiter.
3. **Rapport de qualité des données** : volumétrie, anomalies de format, doublons, trous, incohérences. Chaque constat est chiffré et accompagné de la requête qui le prouve.

Partie C : métriques

1. Instrumentez le service (`prom-client`) et exposez `/metrics` : métriques RED de l'API, parties en cours, rapports reçus par cause et par build, etc. Justifiez chaque métrique, son type et ses labels.
2. Prometheus : scraping, au moins deux recording rules utiles.
3. Expliquez ce qui relève des métriques et ce qui relève des logs dans votre architecture.

Partie D : tableaux de bord

Tableaux de bord Grafana **provisionnés sous forme de code** (datasources et dashboards versionnés) :

- santé du service ;
- performance côté joueur ;
- activité de jeu et intégrité des parties.

Chaque panneau répond à une question écrite en titre ou en description. Un panneau sans question est retiré de la notation.

Partie E : détection de comportements

1. Établissez une typologie des rapports de performance à partir des données (causes, signatures), puis classez automatiquement chaque rapport.
2. Identifiez et caractérisez tout comportement anormal : dégradation liée à une version, population touchée, phénomène récurrent, parties ou clients suspects, données falsifiées.
3. Traduisez vos détections en règles d'alerte (Prometheus, ruler Loki ou Grafana Alerting) avec seuils justifiés. Discutez les faux positifs et faux négatifs.

Partie F : rapport d'incident

Rédigez un postmortem répondant aux deux questions de la direction : chronologie, cause, population touchée, preuves (requêtes et captures), actions correctives et préventives.

4. Livrables

- Dépôt Git avec historique de commits régulier sur les 4 jours, pipeline visible et fonctionnel.
- `docker compose up` lance l'ensemble ; README d'installation.
- Rapport PDF (15 pages maximum) : parties B3, C3, E, F.
- Export JSON des dashboards et captures d'écran.

- Vidéo de démonstration (4 minutes maximum).

5. Barème

Partie	Points
A. CI/CD et déploiement	4
B. Collecte de logs et qualité des données	4
C. Métriques	2
D. Tableaux de bord	3
E. Détection de comportements	4
F. Rapport d'incident	2
Qualité globale	1
Total	20